

Agentic Search: When the Agent Decides What to Search

Ashish Tiwari

Principal Solutions Architect (Search AI Specialist)

September 25, 2026

Agenda

- | | |
|----|--|
| 01 | Why search breaks |
| 02 | The autonomy ladder |
| 03 | Built natively in Elastic |
| 04 | Live demo |
| 05 | What it costs, and where to start |

“SOMETHING TO KEEP MY COFFEE HOT ON A LONG DRIVE”

Same index. Same Elasticsearch. Only one thing changed: who decides what to search for.

Agentic search

Understands the **need**: a travel mug or thermos. Rewrites the query, picks the category, then runs hybrid search with reranking.

Keyword search

Matches the **words**: coffee beans, coffee makers, anything that says “coffee”. It did exactly what we asked.

Why search breaks



USERS DESCRIBE NEEDS, NOT QUERIES

Traditional search assumes the user knows **what to ask** and **where to look**. Four real queries from the demo catalog show where that breaks.

Needs, not keywords

“keep my coffee hot” means a thermos, not coffee.

Hidden constraints

“nike running shoes under 100 dollars” is a brand, a category and a price filter.

Messy input

“samsng phone”, “sasta laal lipstick”: typos and Hinglish.

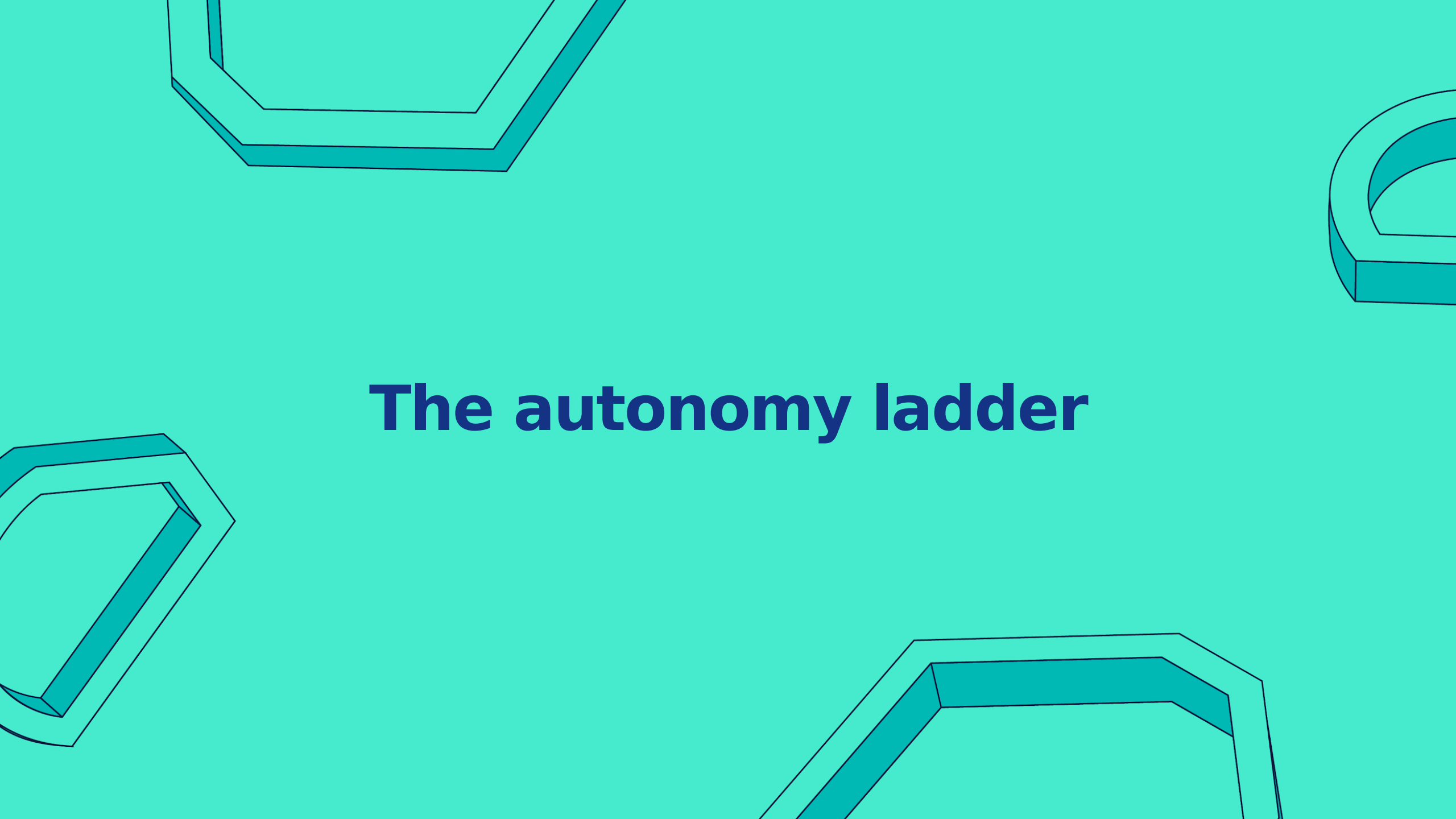
Wrong place to look

“can I return perfume?” is a policy question, not a product search.

THE IDEA IN ONE LINE

“The agent decides what to search. Elasticsearch still decides what's relevant.”

The design principle for the rest of this talk

The background features several abstract, three-dimensional geometric shapes in shades of teal and light blue. These shapes are scattered around the central text, including a large L-shaped structure at the top left, a curved shape at the top right, a hexagonal-like structure at the bottom left, and a stepped structure at the bottom right.

The autonomy ladder

“Agentic” is a ladder, not a switch

Each rung hands one more decision from the user to the model. Pick the rung per use case.

L1 Hybrid

BM25 + vectors + rerank. The engine decides relevance.

L2 Understand

The LLM picks filters, fixes typos, rewrites.

L3 Route

The LLM picks where to search, retries on zero hits.

L4 Agent

The agent picks which searches and how many, then combines.

GIVE EACH JOB TO THE RIGHT COMPONENT

The LLM is good at judgement and bad at retrieval. Elasticsearch is the opposite. Keep them in their lanes.



LLM for judgement: intent, filters, rewrite, route, as structured JSON.



Elasticsearch for retrieval: hybrid search, RRF, reranking, aggregations.



Workflows for control: deterministic steps, guardrails, a trace for every run.

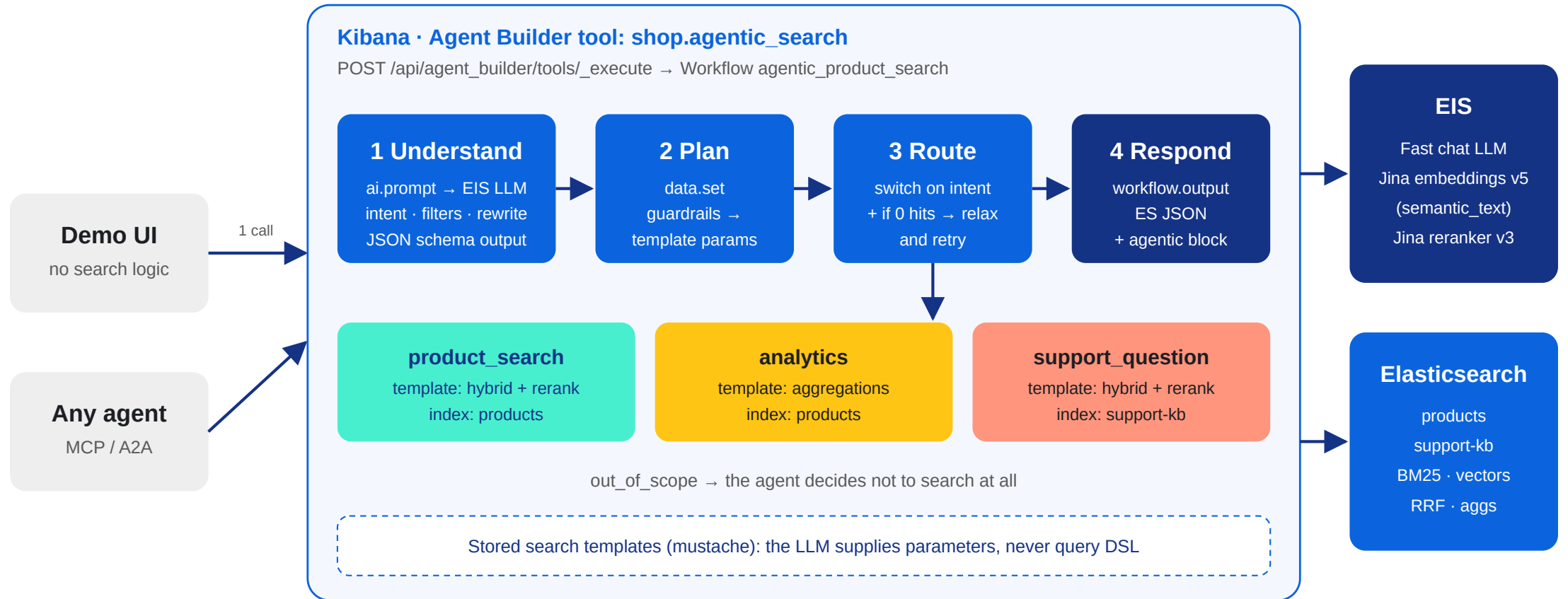


Agent Builder for reach: the same pipeline as an API, a chat agent, MCP and A2A.

The background is a solid dark blue. It features several abstract, 3D-style geometric shapes in a lighter blue color. These shapes include a large 'C' or 'G' shape in the top left, a partial 'C' shape in the top right, a hexagonal ring-like shape in the bottom left, and a large, open-bottomed polygonal shape in the bottom right. The text is centered in the middle of the image.

Built natively in Elastic

One API call, one workflow, zero glue code



The LLM fills a schema. It never writes query DSL.

1 · AI.PROMPT OUTPUT (JSON SCHEMA)

```
{
  "intent": "product_search",
  "search_text": "running shoes",
  "categories": ["mens-shoes", "womens-shoes"],
  "brands": ["Nike"],
  "price_max": 100,
  "sort_by": "relevance",
  "reasoning": "Nike running shoes, $100 cap"
}
```

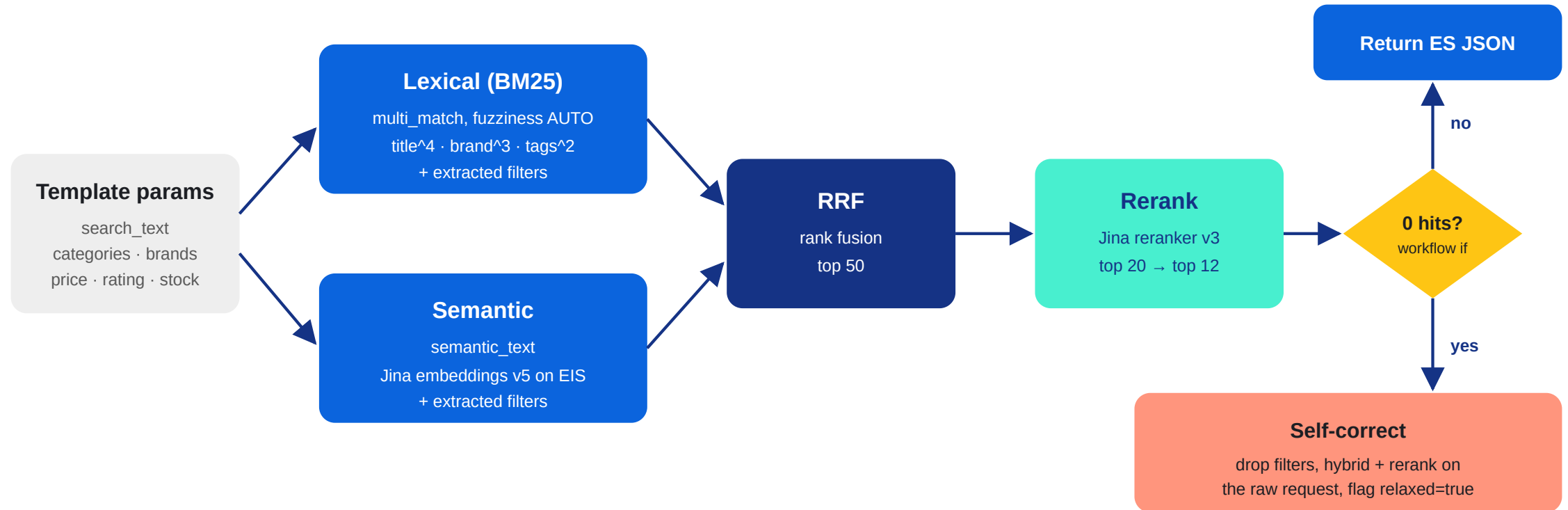
Grounded: the prompt carries the **real** category and brand list from the index, so “samsng” becomes Samsung.

2 · STORED SEARCH TEMPLATE (MUSTACHE)

```
"filter": [
  { "bool": { "should": [
    {{#brands}}{{ "term": { "brand": "{{.}}" } },{{/brands}}
    {{^brands}}{{ "match_all": {} } },{{/brands}}
    { "match_none": {} } ] } },
    {{#price_max}}
    { "range": { "price": { "lte": {{price_max}} } } },
    {{/price_max}}
    { "match_all": {} }
  ]
}
```

The query is written and tested by us. The model only supplies parameters, so a bad answer can't produce a bad query.

Hybrid retrieval, reranking, self-correction



One stored template, two branches: text + relevance → hybrid + rerank; filters only or sort by price/rating → filtered BM25 + sort

SIX ELASTIC BUILDING BLOCKS, NO EXTRA SERVICES

semantic_text on EIS

Jina v5 embeddings created at ingest. No model hosting.

Retrievers

rrf fuses BM25 and semantic results in one request.

Reranker

text_similarity_reranker with Jina reranker v3.

Search templates

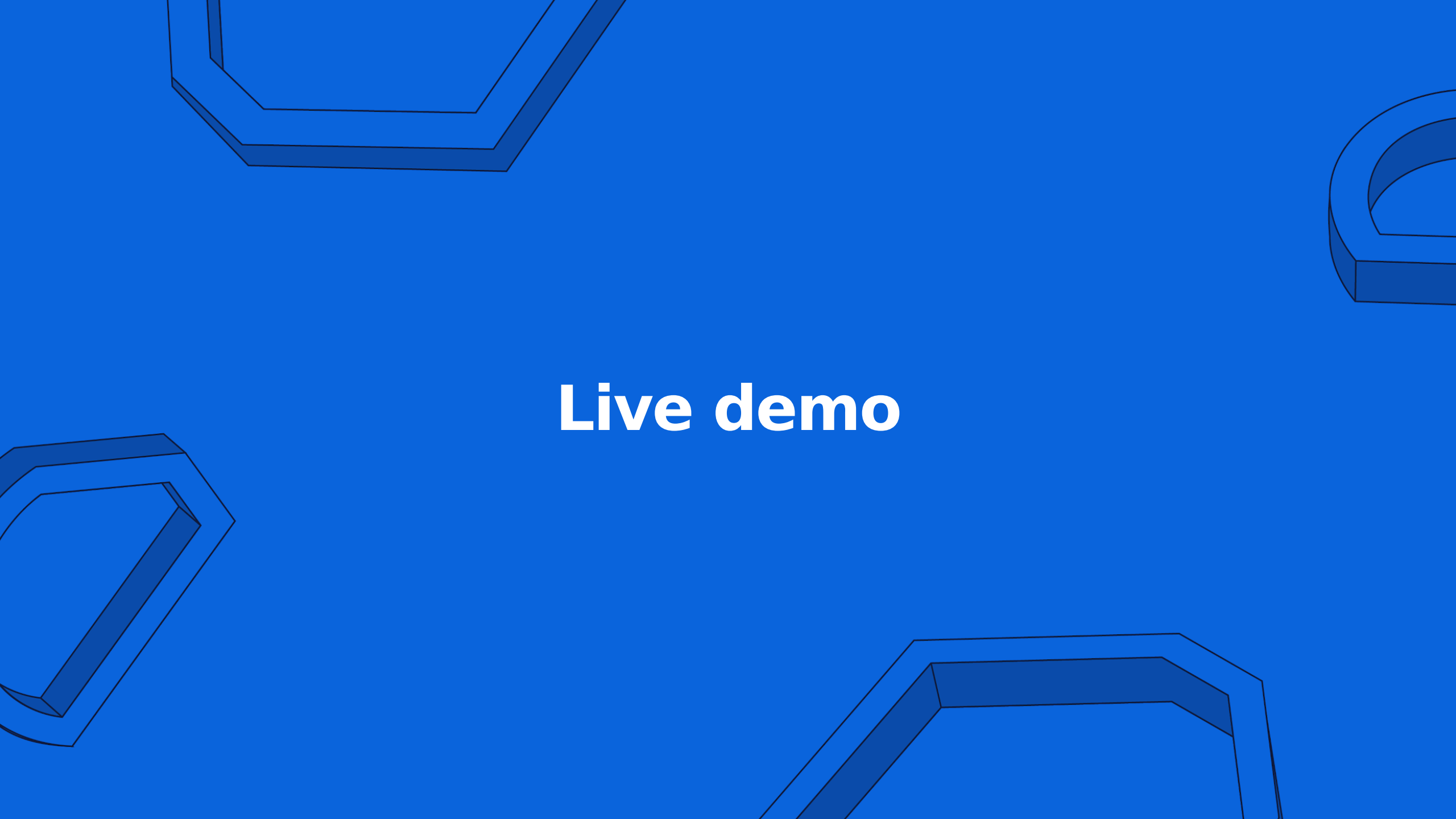
Tested query shapes. The LLM fills parameters only.

Workflows

ai.prompt, switch, if, data.set, workflow.output in YAML.

Agent Builder

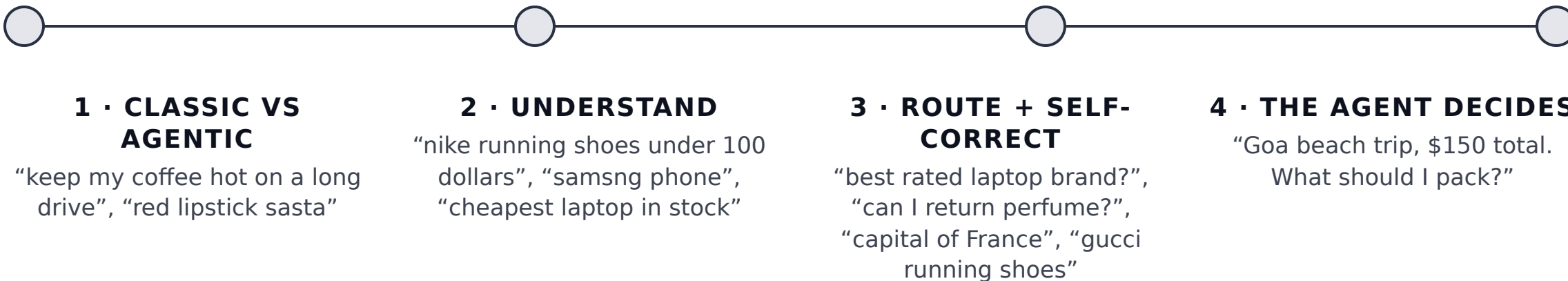
Workflow tools, ES|QL tools, agents, MCP and A2A.

The background is a solid blue color. In the corners, there are abstract, 3D-style geometric shapes. In the top-left, there's a shape resembling a stylized 'L' or a corner of a cube. In the top-right, there's a curved, C-shaped element. In the bottom-left, there's a more complex, multi-faceted geometric form. In the bottom-right, there's a shape that looks like a stylized 'U' or a wide, shallow channel. All these shapes are rendered in a lighter blue color with dark blue outlines, giving them a three-dimensional appearance.

Live demo

Demo run-sheet

ShopSphere: 194 real products with images, 20 help-centre articles, one Elastic Serverless project.



Still Elasticsearch JSON, plus the reason why

WHAT THE API RETURNS

```
{
  "took": 41,
  "hits": {
    "total": { "value": 7 },
    "hits": [
      { "_score": 0.93,
        "_source": { "title": "Nike Air Jordan 1...",
                     "brand": "Nike", "price": 89.99 } }
    ]
  },
  "agentic": {
    "route": "product_search",
    "rewritten_query": "running shoes",
    "relaxed_fallback": false,
    "execution_id": "..."
  }
}
```

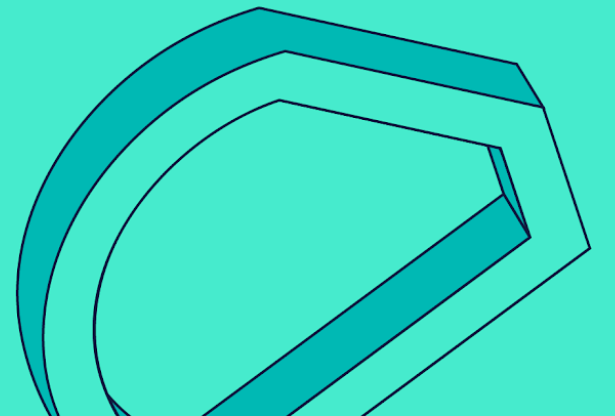
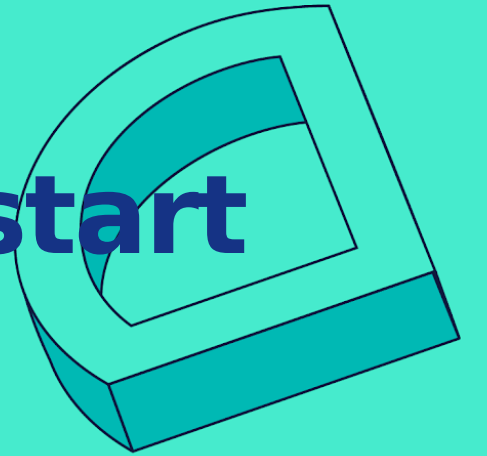
WHY THAT MATTERS

- **Your front-end doesn't change:** same `took`, `hits`, `aggregations`
- **Explainable:** the `agentic` block says what was decided and why
- **Traceable:** every run has an execution trace in Kibana, step by step
- **Reusable:** the same tool serves the UI, the agent, and MCP / A2A clients

L4: the agent decides what, and how often, to search



What it costs, and where to start



Trade-offs, and how we handled them

Concern	What we did in the demo
Latency	One LLM call per search, a small fast model on EIS, <code>reasoning-level: none</code> , catalog list built into the prompt
Wrong answers	JSON schema output, index vocabulary in the prompt, stored templates, zero-hit fallback
Debuggability	Every run has a workflow execution trace; the response carries the decisions
Cost	Agent (L4) only for goals; single searches stay at L2/L3
Lock-in	Standard ES JSON out; the tool is exposed over MCP and A2A

START ON MONDAY

You don't need a new stack. Put a thin agentic layer in front of the index you already have.



1 • Understand

One workflow: ai.prompt → search template on your existing index.



2 • Route

Add a switch for your second index and a zero-hit fallback.



3 • Expose

Wrap it as an Agent Builder tool.
Let agents call it over MCP.

Thank you!

Questions? The demo project: workflows, templates, agent and UI.